14 haftalık yol haritasında 13. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 13

Karakter Dizileri ve Metin İşleme: Güvenli Kodlamaya Giriş

'\0' ile biten char dizileri · güvenli girdi (fgets, snprintf) · string.h ve ctype.h · kendi strlen/strcpy/strcmp · ters çevirme, palindrom, Sezar, kelime sayma, frekans analizi · UTF-8 · argc/argv · tampon taşması (CWE-120/121) ve güvenli kodlama

MODÜL VI · METİN, KAYIT VE KALICILIK · H13–H14

> **HAFTANIN BÜYÜK SORUSU**
>
> *1988'de İnternet'e bağlı bilgisayarların kayda değer bir bölümünü etkileyen solucan, hangi C fonksiyonunun zaafından yararlandı?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ... 3
1.1 Öğrenme çıktıları ... 3
2. Geçen Haftadan: İşaretçiden Karaktere ... 3
3. Kavramsal Anlatım ... 4
3.1 char tekrarı: karakter değişmezi ile string değişmezi ... 4
3.2 '\0' sonlandırıcısı, "boyut + 1" kuralı ve sizeof ↔ strlen ... 5
3.3 Güvenli girdi: scanf'in tuzakları, fgets ve akışlar ... 6
3.4 string.h ve kendi gerçeklememiz ... 7
3.5 Güvenli alternatifler ve snprintf ... 9
3.6 ctype.h ve unsigned char kuralı ... 9
3.7 String ↔ sayı dönüşümü: atoi ve strtol ... 10
3.8 char * ile dolaşma; string dizileri ve işaretçi dizileri ... 10
3.9 Komut satırı argümanları: argc ve argv ... 11
3.10 Karakter kodlaması: ASCII ve UTF-8 ... 12
3.11 Kaputun altında: tampon taşması ve güvenli kodlama ... 14
4. Çözümlü Örnekler: Örnek Merdiveni ... 15
4.1 Örnek 3–4: harf dönüşümü ve karakter sayma (★) ... 15
4.2 Örnek 6–7: ters çevirme ve palindrom (★★) ... 16
4.3 Amiral örnek — Örnek 9 ★★: Sezar şifreleme ve kaba kuvvet ... 17
4.4 Örnek 13–14: kelime sayma ve frekans analizi (★★★) ... 19
4.5 Amiral örnek — Örnek 16 ★★★: tampon taşması ve güvenli sürüm ... 21
5. Hata Avcısı ... 22
6. Mühendis Gözüyle: Güvenli Kodlama, Araçlar ve Taşınabilirlik ... 23
7. Mini-OBS v0.13: Ad–Soyad, İsimle Arama ve Güvenli Girdi ... 23
8. Alıştırmalar ve Atölye ... 25
8.1 Alıştırmalar ... 25
8.2 Atölye etkinlikleri ... 26
9. Kendini Değerlendirme ... 27
10. AP-2 Köprüsü ve Merak Kutusu ... 27
11. Özet, Kavram Haritası ve Terimler ... 27
12. Kaynaklar ... 28

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

1988'de İnternet'e bağlı bilgisayarların kayda değer bir bölümünü etkileyen solucan, hangi C fonksiyonunun zaafından yararlandı?

Bu haftaya kadar sayılarla çalıştık; şimdi **metinle** çalışacağız. Bir ad, bir parola, bir dosya satırı, bir URL — hepsi **karakter dizisidir** (string). C'de string ayrı bir tip değildir: `'\0'` (sayısal değeri 0 olan boş karakter) ile biten bir **char dizisidir**. (Ayrıntılı ders kitabı anlatımı: King, 2008, Bölüm 13; Deitel ve Deitel, 2022, Bölüm 8.) Bu basit sözleşme büyük bir güç verir (metni karakter karakter işleyebiliriz) ama büyük bir sorumluluk da yükler: dizinin sonunu imleyen `'\0'` unutulursa ya da bir tampona sığmayan metin yazılırsa program komşu belleği sessizce bozar. Dönemin güvenlik izleğinin doruğu buradadır: 1988'de İnternet'in kayda değer bir bölümünü etkileyen solucan, tam da böyle bir hatadan — sınır denetimi yapmayan bir girdi fonksiyonundan — yararlandı.

> GERÇEK DÜNYADAN • **2 Kasım 1988: Morris solucanı ve bir fonksiyonun bedeli**
>
> **Morris solucanı**, 2 Kasım 1988 akşamı İnternet'e bırakıldı ve 24 saat içinde o dönemde bağlı olan yaklaşık 60.000 makinenin onda birini, tahminen 6.000 bilgisayarı etkiledi (Federal Bureau of Investigation, 2018). Solucanın yayılma yollarından biri, uzaktaki bir makinede çalışan `fingerd` (kullanıcı bilgisi) hizmetiydi. 4.3BSD'nin `fingerd.c` kaynağında girdi tamponu `char line[512];` olarak bildirilmiş ve `gets(line);` ile dolduruluyordu (Regents of the University of California, 1985); `gets` okuduğu baytların **tampon sınırını denetlemez**. Spafford'ın çözümlemesine göre (Spafford, 1989) solucan bu hizmete 536 baytlık özel olarak hazırlanmış bir dizi gönderdi; tampon taştı ve yığın çerçevesindeki komşu veriler ile çağrının **dönüş adresi** bozuldu; bu da solucanın kendi komutlarını çalıştırmasına olanak verdi. Mekanizma mimariye bağlıydı: bu etki VAX üzerindeki 4 BSD'de işe yaradı, Sun makinelerinde ise yalnızca bir çökmeyle sonuçlandı.
>
> Bu olayın C diline kalıcı bir izi vardır: **gets önce C99'un üçüncü düzeltmesinde (TC3) kullanımdan kaldırıldı, sonra C11 standardından tamamen çıkarıldı** (ISO/IEC, 2018; Carnegie Mellon University Software Engineering Institute, 2016). Bu hafta `gets`'i hiç kullanmayacağız; yerine, okuyacağı bayt sayısını tamponun boyutuyla sınırlayan `fgets`'i kullanacağız. Soru güncelliğini koruyor: tampon taşması bugün de en yaygın güvenlik açığı sınıflarından biridir (CWE-120, CWE-121).

1.1 Öğrenme çıktıları

ÖÇ 13.1 C'de karakter dizisinin '\0' ile sonlanan char dizisi olduğunu bellek düzeyinde açıklar. *(Bölüm 3.2, Örnek 1)*

ÖÇ 13.2 Karakter ve metin girişi/çıkışını güvenli biçimde (fgets, snprintf) yapar. *(Bölüm 3.3, Örnek 2)*

ÖÇ 13.3 string.h ve ctype.h fonksiyonlarını kullanır; temel string fonksiyonlarını (strlen, strcpy, strcmp) kendisi gerçekler. *(Bölüm 3.4, Örnek 5 ve 8)*

ÖÇ 13.4 Karakter düzeyinde metin işleme algoritmaları (sayma, dönüştürme, palindrom, kelime sayma, şifreleme) tasarlar. *(Bölüm 4, Örnek 6–7, 9, 13–14)*

ÖÇ 13.5 Tampon taşması açığının mekanizmasını ve önleme yollarını açıklar. *(Bölüm 3.11 ve 5, Örnek 16)*

2. Geçen Haftadan: İşaretçiden Karaktere

H12'de bir değişkenin **adresini** tutan işaretçileri, `a[i]` ile `*(a + i)` denkliğini ve `malloc`/`free` ile öbekte dinamik bellek yönetmeyi öğrendik. Bu hafta string'ler bu bilgilerin doğrudan uygulamasıdır: bir string bir `char` **işaretçisiyle** dolaşılır (`*(s + i)`), bir isim listesi bir `char` dizisidir, Mini-OBS'nin yeni ad–soyad alanları öbekte `realloc` ile büyür. Önce üç geri çağırma sorusu (`kodlar/quiz/`):

> **TAHMİN ET · Geri çağırma: üç soru**
>
> - **S1 (dizi–işaretçi denkliği).** `int a[] = {10,20,30,40}; int *p = a + 1;` → `a[2]`, `*(a + 2)`, `p[1]` ne yazdırır? (`H13_Q01_dizi_denklik.c`)
> - **S2 (sizeof tuzağı).** `double *d = malloc(4 * sizeof *d);` → `sizeof d` ve `4 * sizeof *d`? (`H13_Q02_malloc_sizeof.c`)
> - **S3 (işaretçi bildirimi).** `int *p, q;` → `sizeof p` ve `sizeof q`? (`H13_Q03_int_p_q.c`)

Yanıtlar (gerçek çıktılar). S1: "30 30 30": `a[i]` ile `*(a + i)` aynı kutudur; bu hafta bir string'i `while (*s != '\0') s++;` ile tam olarak bu denklikle dolaşacağız. S2: "8 32": `sizeof d` işaretçinin 8 baytı, `4 * sizeof *d` ise bloğun 32 baytıdır; aynı tuzak string tamponlarında da (Örnek 16, HA1) karşımıza çıkacak. S3: "8 4": bildirimde `*` ada bağlanır.

Bu haftanın konusu, önceki haftalarda açık bıraktığımız birkaç sorunun da yanıtıdır (Tablo 13.1).

Tablo 13.1
Önceki haftalarla bağlantılar

Hafta	Bağlantı	Bu haftaki karşılığı
H3	`char` = küçük bir tamsayı; ASCII	Harf dönüşümü ASCII aritmetiğiyle: `'a' - 'A' = 32` (Örnek 3)
H6	Basamak biriktirme (`sonuc * 10 + rakam`)	Kendi `atoi`'miz: metinden sayıya (Örnek 10)
H10	Frekans dizisi ve histogram	Harf frekansı ve frekans analiziyle Sezar kırma (Örnek 14)
H10	İki uç tekniği (dizi ters çevirme)	String'i yerinde ters çevirme ve palindrom (Örnek 6–7)
H9	Yığın çerçevesi ve dönüş adresi	Tamponun taşarak dönüş adresini ezmesi (Bölüm 3.11, Örnek 16)
H12	`char *` ile belleği dolaşma; `realloc`	String'leri işaretçiyle dolaşma; Mini-OBS'de ad bloğu (Bölüm 7)

3. Kavramsal Anlatım

3.1 char tekrarı: karakter değişmezi ile string değişmezi

H3'te `char`'ın **küçük bir tamsayı** olduğunu, her karakterin bir **ASCII** koduyla temsil edildiğini görmüştük (`'A'` = 65, `'a'` = 97). Bir **karakter değişmezi** tek tırnakla yazılır (`'A'`) ve tek bir `char`'dır; bir **string değişmezi** çift tırnakla yazılır (`"A"`) ve **iki** bayttır: `'A'` ve sonlandırıcı `'\0'`. İki bildirimi ayırt etmek bu haftanın en önemli ayrımlarından biridir:

H13_K01_string_degismez.c · satır 21–28

```
21      char s[] = "abc";          /* diziye kopya: 4 bayt, değiştirilebilir */
22      const char *p = "abc";           /* değişmezi gösterir: salt okunur say */
23
24      s[0] = 'X';                      /* GÜVENLİ: dizinin kendi baytı */
25      printf("s değişti: %s\n", s);
26
27      /* p[0] = 'X';  <- TANIMSIZ DAVRANIŞ: değişmezi değiştirme (HA6) */
28      printf("p (değişmez): %s\n", p);
```

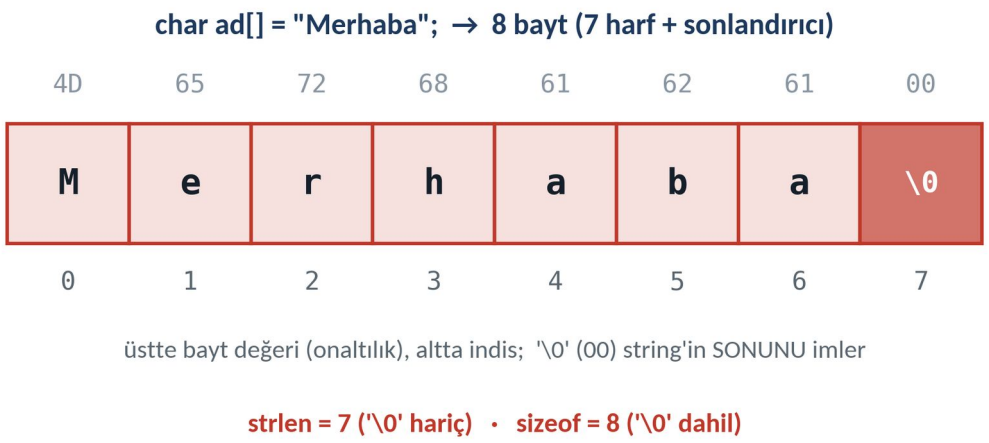
`char s[] = "abc"` string'i bir **diziye kopyalar**: 4 bayt (`a b c '\0'`), değiştirilebilir. `char *p = "abc"` ise p'yi, bellekte duran **salt okunur bir string değişmezine** yöneltir; `p[0] = 'X'` yazmak **tanımsız davranıştır** (çoğu sistemde çöker, HA6). Gerçek çıktı farkı gösteriyor: "s değişti: Xbc" (dizi değişti),

"sizeof p = 8 (yalnızca işaretçinin boyutu)". GCC, string değişmezlerini `-Wwrite-strings` ile `const char *` sayar ve bu atamayı derleme zamanında da uyarır.

3.2 '\0' sonlandırıcısı, "boyut + 1" kuralı ve sizeof ↔ strlen

Bir string, bellekte art arda duran karakterlerden ve sonunda tek bir `'\0'` baytından oluşur (Şekil 13.1). `'\0'`, string'in **nerede bittiğini** imler; `printf("%s")` ve kütüphane fonksiyonları onu bulana dek ilerler. Bu yüzden **n karakterlik bir string en az n + 1 bayt ister** ("boyut + 1" kuralı): `'\0'` için bir bayt daha. Örnek 1, aynı sözcüğü üç biçimde yazdırır ve baytları onaltılık gösterir:

Şekil 13.1
"Merhaba" bellekte: 7 harf + sonlandırıcı '\0' (00); bayt değeri üstte, indis altta



H13_01_string_tanimlama.c · satır 16–35

```c
    char ad[] = "Merhaba";      /* 7 harf + '\0' = 8 bayt, değiştirilebilir */
    char kisa[6] = "AP-1";          /* 4 harf + '\0' + 1 kullanılmayan bayt */

    printf("%%s ile       : %s\n", ad);

    printf("karakter karakter: ");
    for (int i = 0; ad[i] != '\0'; i++) {   /* '\0' bulunca dur */
        printf("[%c]", ad[i]);
    }
    printf("\n");

    printf("baytlar (onaltılık): ");
    for (size_t i = 0; i < sizeof ad; i++) {
        printf("%02X ", (unsigned char)ad[i]);   /* son bayt 00 = '\0' */
    }
    printf("\n");

    printf("sizeof ad   = %zu bayt\n", sizeof ad);     /* 8: '\0' dahil */
    printf("strlen(ad)  = %zu karakter\n", strlen(ad));/* 7: '\0' hariç */
    printf("sizeof kisa = %zu bayt, strlen = %zu\n", sizeof kisa, strlen(kisa));
```

Gerçek çıktı (son üç satır): "sizeof ad = 8 bayt", "strlen(ad) = 7 karakter", "sizeof kisa = 6 bayt, strlen = 4". **İki ölçüyü karıştırmayın:** `sizeof` derleme zamanında **tipin bayt boyutudur** (dizide tüm dizi, `'\0'` ve kullanılmayan baytlar dahil); `strlen` çalışma zamanında `'\0'` a kadar **karakter** sayar. `ad` için `sizeof` 8, `strlen` 7; `char kisa[6] = "AP-1"` için `sizeof` 6, `strlen` 4 (`H13_K02_sizeof_strlen.c`

aynı farkı dizi ve işaretçi için gösterir). İçte bir `'\0'` varsa string orada erken biter: bu, web sayfasındaki "Kendini sına" quizinin 2. sorusunun konusudur.

3.3 Güvenli girdi: scanf'in tuzakları, fgets ve akışlar

Metin okumanın birden çok yolu vardır ama hepsi eşit güvenli değildir. `scanf("%s", ...)` iki nedenle tehlikelidir (`H13_K03_scanf_tehlike.c`): (1) ilk **boşlukta durur**, bu yüzden "Ayşe Nur" girildiğinde yalnızca "Ayşe" okunur; (2) alan **genişliği verilmezse sınırsız okur** ve tamponu taşırabilir. Genişlik verilerek (`%19s`, 20 baytlık tampon için) taşma önlenir; ama bir **satırın tamamını** güvenle okumanın doğru yolu `fgets`'tir:

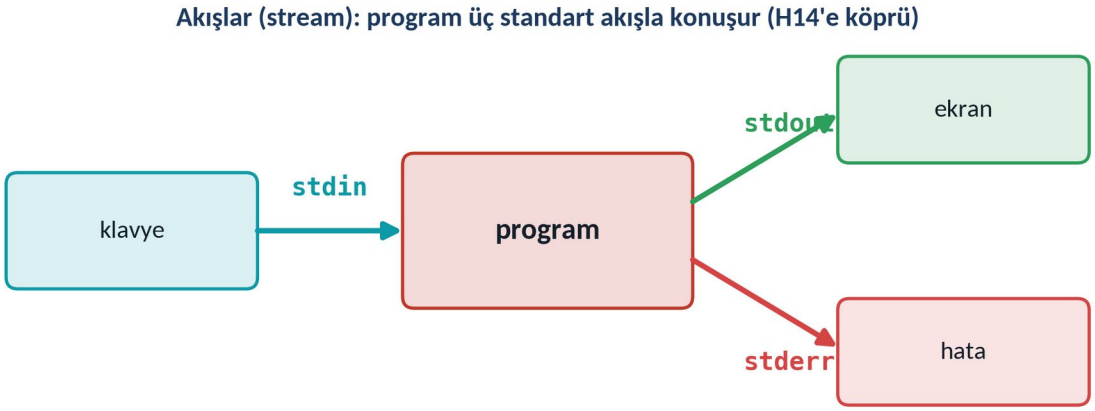
H13_02_fgets_ad_soyad.c · satır 22–26

```
22      if (fgets(ad, sizeof ad, stdin) == NULL) {   /* EOF ya da hata */
23          printf("Girdi okunamadı.\n");
24          return 1;
25      }
26      ad[strcspn(ad, "\n")] = '\0';                /* sondaki '\n'i sil */
```

`fgets(ad, sizeof ad, stdin)` en çok `sizeof ad - 1` bayt okur, geri kalanı bırakır ve sona her zaman `'\0'` koyar: **taşmaz**. Tek inceliği, satır sonundaki `'\n'`i de tampona koymasıdır; bunu `ad[strcspn(ad, "\n")] = '\0';` ile sileriz (`strcspn`, ilk `'\n'`e kadar olan karakter sayısını verir). `'\n'`i silmemek sık bir hatadır: `strcmp(cevap, "evet")` hiç eşleşmez (HA4). Ayrıca `scanf("%d")` ile `fgets` karıştırıldığında `scanf`'in bıraktığı `'\n'` bir sonraki `fgets`'i boş satır gibi okutur (`H13_K04_fgets_scanf_tuzak.c`): araya kalan satırı temizleyen bir `getchar` döngüsü konur.

> KAPUTUN ALTINDA · **Akışlar: stdin, stdout, stderr (H14'e köprü)**
>
> Bir C programı dış dünyayla üç **standart akış** üzerinden konuşur (Şekil 13.2): **stdin** (standart girdi, genellikle klavye; `fgets`/`scanf` buradan okur), **stdout** (standart çıktı, genellikle ekran; `printf` buraya yazar) ve **stderr** (standart hata; hata iletileri `fprintf(stderr, ...)` ile buraya yazılır, böylece çıktı bir dosyaya yönlendirilse bile hatalar ekranda kalır). `fgets(..., stdin)`, `./program < girdi.txt` ile bir dosyadan da beslenebilir: akış, kaynağından bağımsız bir bayt dizisidir. Bu kavramı H14'te `FILE *` ile dosyalara genişleteceğiz.

Şekil 13.2
Üç standart akış: program stdin'den okur, stdout'a ve stderr'e yazar



fgets stdin'den okur; printf stdout'a, fprintf(stderr,...) hata iletisi yazar

3.4 string.h ve kendi gerçeklememiz

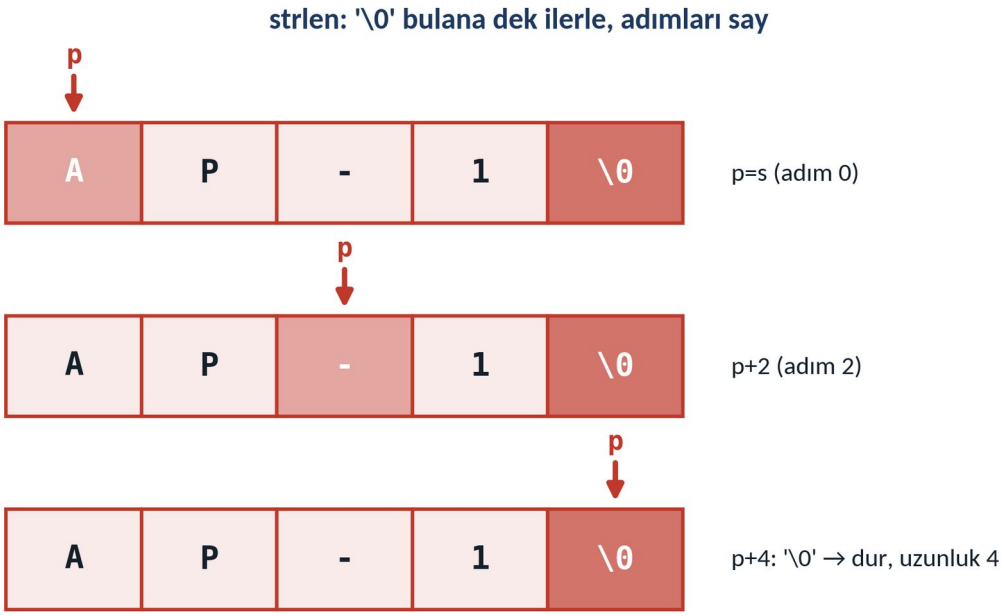
`string.h`, metinle çalışmanın temel araç kutusudur. En sık kullanılanlar Tablo 13.2'dedir; `H13_K05_string_h_tur.c` hepsini bir arada gösterir (gerçek çıktı: `strchr('d')` konum 9, `strstr("dünya")` konum 9).

Tablo 13.2
string.h: en sık kullanılan fonksiyonlar

Fonksiyon	Ne yapar?	Dikkat
`strlen(s)`	'\0'a kadar karakter sayısı	'\0' hariç; `size_t` döner
`strcpy(h, k)` / `strncpy`	k'yı h'ye kopyalar / en çok n bayt	`strncpy` n bayttan uzun kaynakta `'\0'` koymayabilir
`strcat(h, k)` / `strncat`	k'yı h'nin sonuna ekler / en çok n bayt	hedef yeterince büyük olmalı; `strncat`'te n EKLENECEK bayttır
`strcmp(a, b)`	sözlük sırasına göre karşılaştırır	yalnızca dönüşün İŞARETİ anlamlıdır
`strchr(s, c)` / `strstr(s, alt)`	karakterin / alt string'in ilk konumu	bulamazsa NULL döner (işaretçi)
`strtok(s, ayr)`	s'yi ayraçlardan parçalara böler	s'yi DEĞİŞTİRİR; durum tutar; ardışık ayracı tek sayar

Bu fonksiyonların çoğunu kendimiz yazabiliriz; iç işleyişlerini görmek, onları doğru kullanmanın en iyi yoludur. **strlen'in sırrı** "`'\0'` bulana dek ilerle, adımları say"dır (Şekil 13.3); indisle ya da işaretçiyle (`p - s`, H12) yazılır:

Şekil 13.3
strlen: işaretçi '\0' bulana dek ilerler, aradaki adım sayısı uzunluktur (Örnek 5)



H13_05_strlen_kendimiz.c · satır 26–33

```c
size_t benim_strlen_isaretci(const char *s)
{
    const char *p = s;
    while (*p != '\0') {
        p++;
    }
    return (size_t)(p - s);
}
```

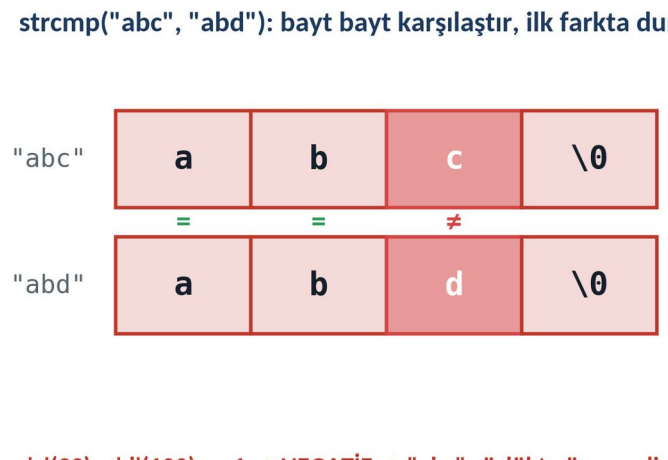
Gerçek çıktı: ""Merhaba" : indis 7, işaretçi 7, strlen 7", ""AP-1" : indis 4, işaretçi 4, strlen 4", """ : indis 0, işaretçi 0, strlen 0". Boş string yalnızca `'\0'` tır: uzunluğu 0. **strcmp** iki metni bayt bayt karşılaştırır, ilk farklı baytta durur ve farkın (unsigned char olarak) İŞARETİNİ döndürür (Şekil 13.4): negatifse ilk metin sözlükte önce gelir, 0 ise eşittir. `strcmp`'nin kesin sayısı taşınabilir değildir; yalnızca işareti kullanılır (ISO/IEC, 2018, 7.24.4.2). Örnek 8'deki kendi `benim_strcpy` ve `benim_strcmp`'miz standart sürümlerle aynı sonucu verir: strcmp("abc","abd") negatif, strcmp("Zeki","Ali") pozitiftir.

H13_08_strcpy_strcmp_kendimiz.c · satır 28–35

```c
int benim_strcmp(const char *a, const char *b)
{
    size_t i = 0;
    while (a[i] != '\0' && a[i] == b[i]) {
        i++;
    }
    return (unsigned char)a[i] - (unsigned char)b[i];
}
```

Kendi `strcpy`/`strcmp` gerçeklemeleri K&R'nin klasik örnekleridir (Kernighan ve Ritchie, 1988, Bölüm 5.5).

Şekil 13.4
strcmp("abc", "abd"): ilk farklı bayt ('c' vs 'd') dönüşün işaretini belirler



strncpy tuzağı: `strncpy`, kaynak n bayttan uzunsa sona `'\0'` **koymayabilir**; sonuç `'\0'` ile bitmeyen bir bayt dizisi olur ve sonraki `strlen`/`printf` tampon dışına taşar. Güvenli kalıp, kopyadan sonra son baytı elle `'\0'` yapmaktır (`H13_K06_strncpy_tuzak.c`: “strncat 2 bayt: "AP-1"”). `strtok` bir satırı ayraçlardan parçalara böler ama özgün diziyi değiştirir (ayraç yerlerine `'\0'` yazar) ve durum tutar (ilk çağrı diziyle, sonrakiler NULL ile); ardışık ayraçları tek sayar (`H13_K07_strtok.c`: “Toplam 3 parça (ardışık ayraçlar tek sayıldı).”). `strtok`'u bir string değişmezinde çağırmak **tanımsız davranıştır**; bu sistemde program çöktü (çıkış 139, HA5).

3.5 Güvenli alternatifler ve snprintf

Biçimli metin üretirken `sprintf` tehlikelidir: boyut parametresi yoktur, hedefi taşırabilir. Yerine `snprintf` kullanılır: hedefe en çok `boyut - 1` bayt yazar ve sona `'\0'` koyar (taşmaz). Dönüş değeri “tam sığsaydı kaç bayt gerekirdi”dir; **boyuta eşit ya da büyükse kesilme olmuştur** ve bu saptanabilir (`H13_K08_snprintf.c`: “"23:Ali" (gerekli 6, boyut 8, sığdı)”, “"7:Abdul" (gerekli 10, boyut 8, KESİLDİ)”). Bu haftanın güvenli kodlama ilkesi tek cümleyle şudur: **her yazma, hedef tamponun boyutuyla sınırlanmalıdır** (SEI CERT STR31-C; Seacord, 2013).

> MERAK KUTUSU • **Annex K ve strcpy_s**
>
> C11, isteğe bağlı bir ek (Annex K) ile `strcpy_s`, `gets_s` gibi sınır denetimli sürümler tanımlar. Ancak bunlar **isteğe bağlıdır**: yalnızca `__STDC_LIB_EXT1__` tanımlıysa bulunurlar ve GCC/glibc bunları gerçeklemez (ISO/IEC, 2018, Annex K). Taşınabilir güvenli kod bu yüzden `strcpy_s`'ye değil, boyut sınırlı standart fonksiyonlara (`snprintf`, boyut parametresiyle `fgets`) ve elle denetime dayanır.

3.6 ctype.h ve unsigned char kuralı

`ctype.h`, karakterleri **sınıflandıran** (`isalpha`, `isdigit`, `isspace`, `isupper` …) ve **dönüştüren** (`toupper`, `tolower`) fonksiyonları verir. Kritik bir kural vardır: bu fonksiyonların argümanı önce `unsigned char`'a çevrilmelidir. Çünkü `char` çoğu derleyicide **işaretlidir**; 0x80 ve üzeri bir bayt

(örneğin bir UTF-8 devam baytı) negatif görünür ve negatif bir değeri (EOF dışında) `ctype` fonksiyonuna vermek **tanımsız davranıştır** (SEI CERT STR37-C). `H13_K09_ctype.c` bunu somutlaştırır: `(char)0xC3` bu derleyicide `-61` (negatif), `(unsigned char)` ise `195`'tir. Bu yüzden bu haftanın bütün örneklerinde `isalpha((unsigned char)c)` biçimini kullanıyoruz.

3.7 String ↔ sayı dönüşümü: atoi ve strtol

Bir metni sayıya çevirmek, H6'daki **basamak biriktirme** deseninin (`sonuc = sonuc * 10 + (c - '0')`) metin girdisine uygulanmasıdır. Örnek 10'daki kendi `benim_atoi`'miz baştaki boşlukları atlar, işareti okur, rakam oldukça biriktirir (gerçek çıktı: `"   -42abc"` → -42):

H13_10_atoi_kendimiz.c · satır 24–34

```
24      int isaret = 1;
25      if (s[i] == '+' || s[i] == '-') {        /* isteğe bağlı işaret */
26          isaret = (s[i] == '-') ? -1 : 1;
27          i++;
28      }
29      int sonuc = 0;
30      while (isdigit((unsigned char)s[i])) {   /* rakam oldukça biriktir */
31          sonuc = sonuc * 10 + (s[i] - '0');   /* H6 deseni: karakter -> sayı */
32          i++;
33      }
34      return isaret * sonuc;
```

Ama `atoi`'nin ciddi bir eksiği vardır: **hatayı bildiremez.** `"abc"` için 0, `"123abc"` için 123 döner; `"0"` ile hatayı ayırt edemezsiniz, taşmada davranışı tanımsızdır. Hata denetimi gereken yerde `strtol` kullanılır:

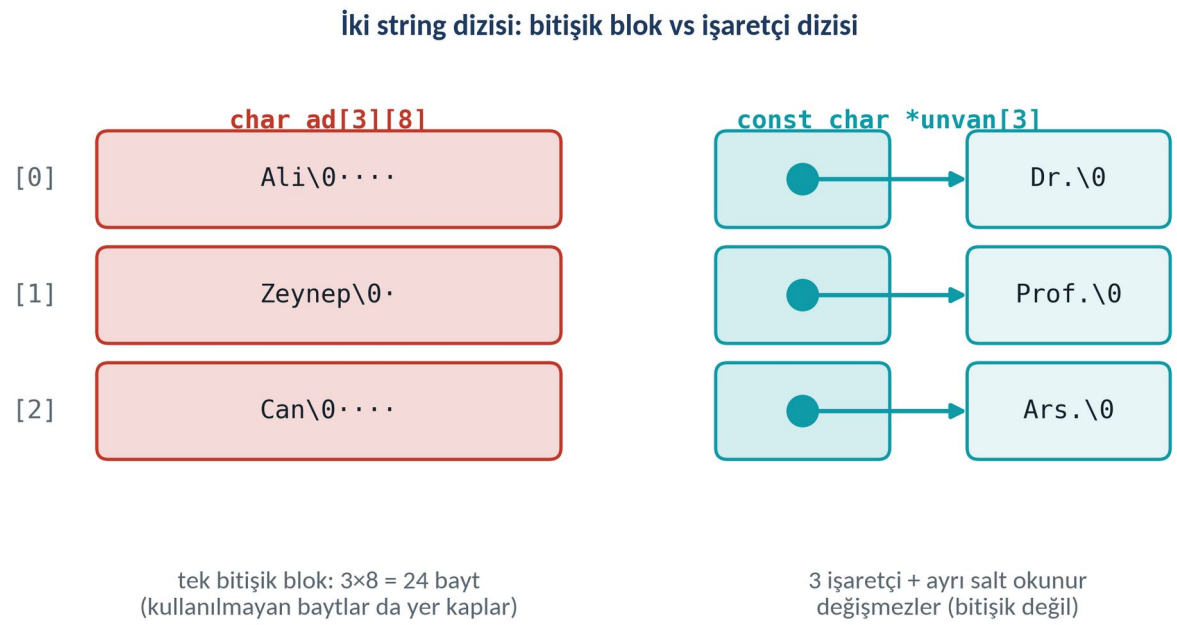
- `endptr`: `strtol(metin, &son, 10)` sayıdan sonra NEREDE durduğunu `son`'a yazar; `*son == '\0'` ise satırın tamamı sayıdır, `son == metin` ise hiç rakam yoktu.
- `errno` / `ERANGE`: çağrıdan önce `errno = 0;` yapılır; sonuç `long`'a sığmazsa `errno` `ERANGE` olur.
- `taban`: 10, 16 ya da 0 (önekten çıkarım: `0x` → onaltılık).

`H13_K10_strtol.c` beş durumu gösterir: “"2026": 2026 (taban 10, tamamı sayı)”, “"12abc": sayı 12, kalan "abc" (tamamı sayı değil)”, “"99999999999999999999": taşma (ERANGE)”. Mini-OBS'nin güvenli girdi fonksiyonları (Bölüm 7) ve Örnek 15'teki hesap makinesi `strtol`/`strtoll` ile bütün girdileri denetler.

3.8 char * ile dolaşma; string dizileri ve işaretçi dizileri

Bir string, bir `char` işaretçisiyle dolaşılır: `for (const char *p = s; *p != '\0'; p++)` her adımda bir sonraki karaktere gider (H12 işaretçi aritmetiği). Birden çok string tutmanın iki yolu vardır (Şekil 13.5):

Şekil 13.5
*char ad[N][M] (tek bitişik blok) ile const char *unvan[] (işaretçi dizisi)*



`char ad[N][M]` N satırlık, her satır M baytlık **tek bitişik bloktur** (H11 bağlantısı: `ad[i]` i. satırın adresi; satırlar arasında M bayt vardır). Satırlar **değiştirilebilir**, ama kullanılmayan baytlar da yer kaplar. `const char *unvan[]` ise bir **işaretçi dizisidir**: her eleman ayrı (ve salt okunur) bir string değişmezini gösterir; metinler bitişik değildir. Örnek 11 ikisini de kullanır, en uzun ismi `strlen` ile bulur ve `strcmp` ile doğrusal tarama yapar (gerçek çıktı: “En uzun isim: Abdullah (8 karakter)”, “Arama "Zeynep": 3. sırada bulundu.”). Bu doğrusal tarama, Mini-OBS'nin isimle aramasının (Bölüm 7) temelidir:

H13_11_isim_listesi.c · satır 23–31

```
23  int isim_bul(char ad[][M], int n, const char *aranan)
24  {
25      for (int i = 0; i < n; i++) {
26          if (strcmp(ad[i], aranan) == 0) {
27              return i;
28          }
29      }
30      return -1;
31  }
```

3.9 Komut satırı argümanları: argc ve argv

Şimdiye dek `main(void)` yazdık; `main`'in parametreleri de olabilir: `int main(int argc, char *argv[])`. `argc`, komut satırı argümanı sayısıdır (program adı dahil); `argv`, argümanları gösteren string işaretçisi dizisidir. `argv[0]` program adıdır, `argv[argc]` her zaman `NULL`'dır (ISO/IEC, 2018, 5.1.2.2.1). Örnek 15'teki hesap makinesi `./hesap 12 x 5` ile çağrılır (çarpma için `*` kabukta özel olduğundan `x` kullanıyoruz) ve argümanları `strtol` ile denetler: metnin TAMAMI sayı değilse ya da taşma varsa reddedilir.

H13_15_argv_hesap_makinesi.c · satır 22–32

```c
static bool tamsayi_coz(const char *metin, long *sonuc)
{
    char *son;
    errno = 0;
    long deger = strtol(metin, &son, 10);
    if (son == metin || *son != '\0' || errno == ERANGE) {
        return false;         /* rakam yok, artık karakter ya da taşma */
    }
    *sonuc = deger;
    return true;
}
```

GERÇEK ÇIKTI (ARGC/ARGV)

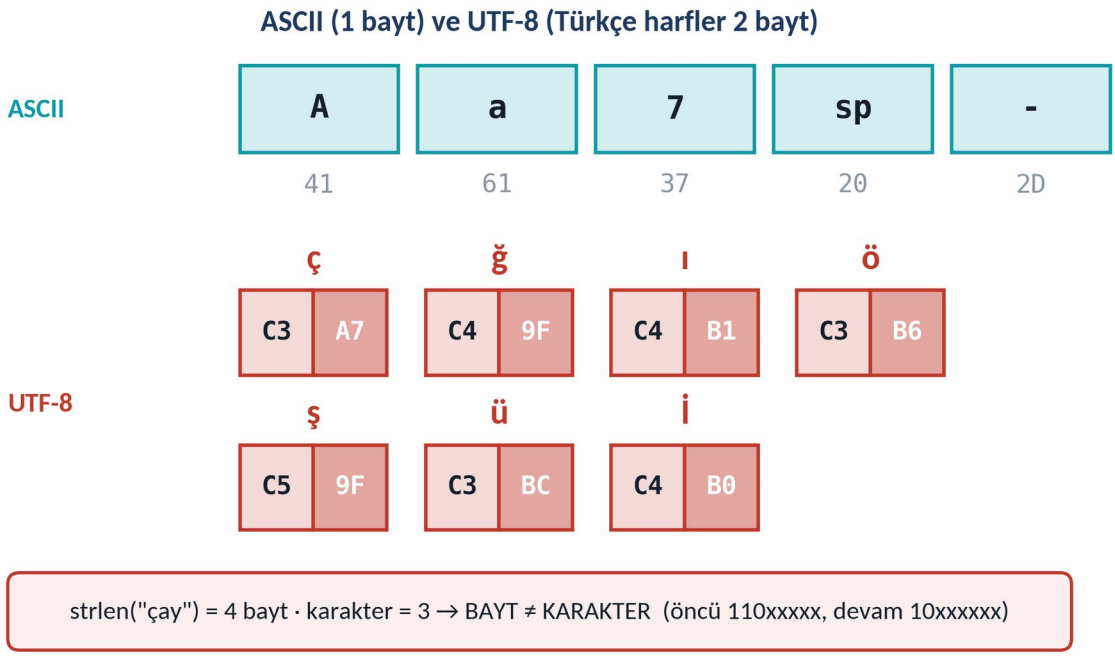
```
$ ./hesap 12 x 5      # çarpma
argc = 4; argv[0] (program adı) = "./hesap"
argv[argc] == NULL mı? evet
12 x 5 = 60

$ ./hesap 100 - 58      # çıkarma
argc = 4; argv[0] (program adı) = "./hesap"
argv[argc] == NULL mı? evet
100 - 58 = 42
```

3.10 Karakter kodlaması: ASCII ve UTF-8

ASCII, her karakteri **tek bir baytla** (0–127) temsil eder: İngiliz alfabesi, rakamlar ve temel noktalama. Ama Türkçe harfler (ç, ğ, ı, ö, ş, ü, İ) ASCII'de yoktur. Modern metin **UTF-8** ile kodlanır: ASCII karakterleri yine tek bayt, Türkçe harfler ise **iki bayt** kaplar (Şekil 13.6). GCC'nin varsayılan kaynak ve yürütme kodlaması UTF-8'dir; bu yüzden kaynak dosyamız UTF-8 kaydedildiğinde `"çay"` kaynakta da bellekte de üç karakter ama **dört bayttır**.

Şekil 13.6
ASCII (1 bayt) ve UTF-8 (Türkçe harfler 2 bayt); öncü bayt 110xxxxx, devam baytı 10xxxxxx



Bayt ≠ karakter. `strlen` BAYT sayar: `H13_12_utf8_deneyi.c` "'çay": 4 bayt, 3 karakter" yazar (strlen 4, karakter 3). Karakter sayısı için devam baytları (`10xxxxxx`) sayılmaz:

H13_12_utf8_deneyi.c · satır 20–29

```
20  size_t utf8_karakter_sayisi(const char *s)
21  {
22      size_t sayi = 0;
23      for (; *s != '\0'; s++) {
24          if (((unsigned char)*s & 0xC0) != 0x80) {
25              sayi++;
26          }
27      }
28      return sayi;
29  }
```

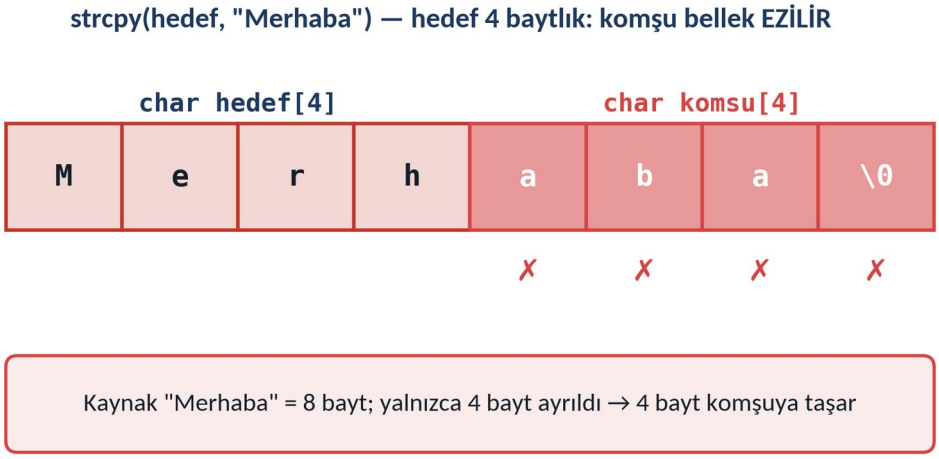
Gerçek çıktıya göre "İSMAİL" 8 bayt / 6 karakter, "ğışöüç" 12 bayt / 6 karakterdir. Bu ayrım pratik sonuçlar doğurur: bir `char[32]` tampon 31 ASCII karakterini ama yalnızca 15 Türkçe harfi alır; `printf`'in `%-24s` hizalaması BAYT saydığı için Türkçe adlarda sütunları kaydırır (Mini-OBS `hizali_yaz`, Bölüm 7).

> HATA AVCISI · **"Türkçe I sorunu"**
>
> `toupper` ve `tolower` tek bir BAYT alır ve varsayılan "C" yerel ayarında (program `setlocale` çağırmadıkça bu ayar geçerlidir) İngilizce kurala göre çalışır: `toupper('i')` `'I'` verir (`H13_K12_turkce_toupper.c`); başka bir yerel ayarda (ör. tr_TR) sonuç farklı olabilir. Oysa Türkçede `'i'`nin büyüğü `'İ'`, `'ı'`nın büyüğü `'I'`dır. Dahası, çok baytlı bir Türkçe harfin baytlarını ayrı ayrı `toupper`'a vermek harfi bozar (0xC3/0xA7 değişmeden kalır). Doğru Türkçe büyütme, UTF-8 farkında bir **eşleme tablosu** gerektirir; Mini-OBS'nin `ad_normallestir` fonksiyonu tam olarak bunu yapar (Bölüm 7). Windows konsolunda Türkçe karakterlerin düzgün görünmesi için `chcp 65001` (UTF-8 kod sayfası) gerekir (H1).

3.11 Kaputun altında: tampon taşması ve güvenli kodlama

Bir string, boyutundan daha uzun bir metinle doldurulursa **komşu belleğe taşar** (Şekil 13.7). `strcpy(hedef, kaynak)` kaynağı `'\0'` dahil kopyalar; hedef kaynaktan küçükse fazla baytlar hedefin ötesindeki belleği ezer — bu **tampon taşmasıdır** (buffer overflow).

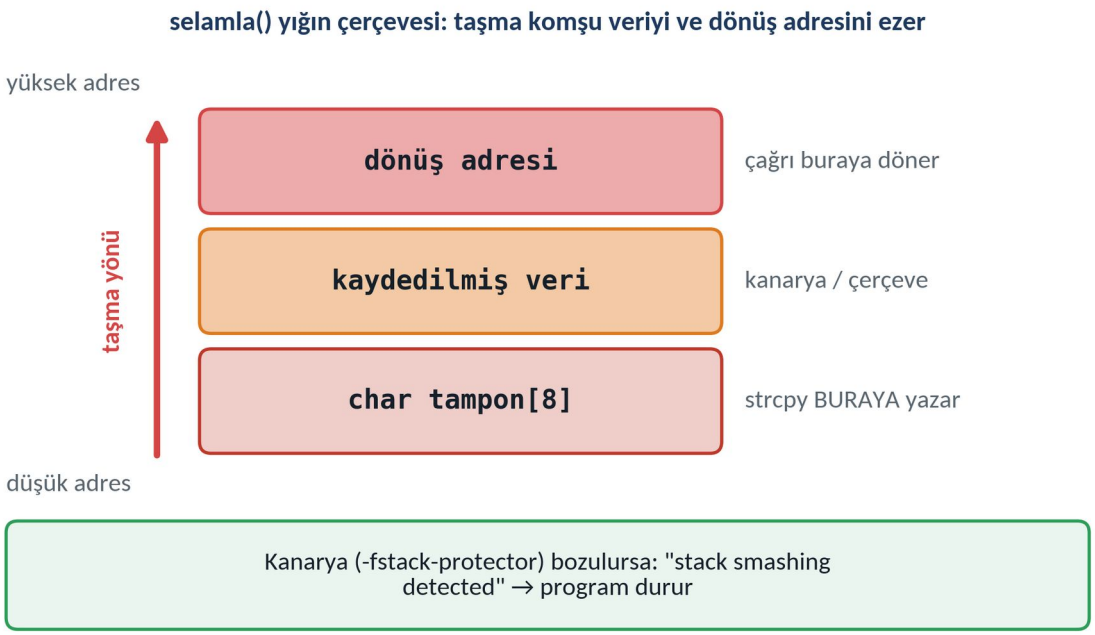
Şekil 13.7
strcpy taşması: 8 baytlık kaynak 4 baytlık hedefe kopyalanınca komşu dizi ezilir



Güvenli: strncpy/snprintf + boyut denetimi (STR31-C)

Hata özellikle **yığında** tehlikelidir (Şekil 13.8; H9'daki yığın çerçevesiyle bağlantılı kavramsal diyagram). Bir fonksiyonun yerel tamponu, yığın çerçevesinde kaydedilmiş değerlerin ve çağrının **dönüş adresinin** yanında durur. Tampon yukarı doğru taşarsa önce komşu veriler, sonra dönüş adresi bozulur; dönüş adresi bozulan bir program ya çöker ya da öngörülemez biçimde davranır. Morris solucanının `fingerd`'de yararlandığı mekanizma tam olarak buydu (Bölüm 1).

Şekil 13.8
Yığın çerçevesinde tampon taşması: taşan baytlar komşu veriyi ve dönüş adresini ezer (kavramsal)



Önleme üç katmanlıdır ve bu haftanın güvenli kodlama ilkelerini oluşturur. (1) **Boyut sınırlı girdi/kopya:** `gets` yerine `fgets`, `strcpy` yerine boyut denetimli kopya, `sprintf` yerine `snprintf` (STR31-C). (2) **Derleyici koruması:** modern derleyiciler yığına bir **kanarya** (canary) değeri koyar (`-fstack-protector`); dönüş anında kanarya bozulmuşsa program "stack smashing detected" iletisiyle **durdurulur** (Örnek 16'da bu derleyicide gerçekten böyle olur; mekanizma için Bryant ve O'Hallaron, 2016, Bölüm 3.10.3–3.10.4). (3) **Araçla doğrulama:** `-fsanitize=address` (AddressSanitizer) her bellek erişimini denetler ve taşmayı ilk olduğu satırda raporlar. Bu kurallar, CWE-120 (Buffer Copy without Checking Size of Input) ve CWE-121 (Stack-based Buffer Overflow) açık sınıflarını ve SEI CERT C string kurallarını (STR30-C değişmezi değiştirme, STR31-C yeterli yer, STR37-C `unsigned char`) doğrudan karşılar (Carnegie Mellon University Software Engineering Institute, 2016; Seacord, 2013). Örnek 16 ve Hata Avcısı bunların hepsini gerçek araç çıktılarıyla gösterir.

4. Çözümlü Örnekler: Örnek Merdiveni

Örnekler kolaydan zora sıralıdır (`kodlar/H13_01_…` – `H13_16_…`); her biri bir yeni öge ekler. Kasıtlı taşan Örnek 16 dışında hepsi **sıfır uyarıyla** derlenir ve (girdi okuyanlar) ASan + UBSan altında temiz çalışır; çıktılar gerçektir. Örnek 1, 2, 5, 8, 10, 11, 12 ve 15'in kilit kesitleri Bölüm 3'te ilgili kavramla birlikte verildi; burada **Örnek 9 (Sezar)** ve **Örnek 16 (tampon taşması)** tam şablonla, diğerleri kilit kesitleriyle verilir.

4.1 Örnek 3–4: harf dönüşümü ve karakter sayma (★)

Örnek 3 — büyük/küçük harf. Her ASCII küçük harf, `'a' - 'A' = 32` kadar kaydırılarak büyütülür (H3 ASCII aritmetiği); aynı iş `ctype.h`'nin `toupper`'ıyla da yapılır. İki yöntem ASCII harfler için aynıdır (gerçek çıktı: "ASCII : AP-1 DERSI"):

H13_03_harf_donusturme.c · satır 25–30

```
25      for (size_t i = 0; metin[i] != '\0'; i++) {
26          char c = metin[i];
27          /* yalnızca ASCII küçük harfleri FARK kadar yukarı kaydır */
28          ascii[i] = (c >= 'a' && c <= 'z') ? (char)(c - FARK) : c;
29          ctyp[i] = (char)toupper((unsigned char)c);   /* unsigned char ZORUNLU */
30      }
```

Örnek 4 — karakter sayma. Bir satır `fgets` ile okunur, her bayt `(unsigned char)` ile `ctype`'a verilerek ünlü, ünsüz, rakam, boşluk ve diğer olarak sınıflandırılır (“Merhaba C 2026!” → Harf : 8 (ünlü 3, ünsüz 5), rakam 4, boşluk 2, diğer 1):

H13_04_karakter_sayma.c · satır 33–48

```
33      for (size_t i = 0; satir[i] != '\0'; i++) {
34          unsigned char c = (unsigned char)satir[i];
35          if (isalpha(c)) {
36              if (unlu_mu(c)) {
37                  unlu++;
38              } else {
39                  unsuz++;
40              }
41          } else if (isdigit(c)) {
42              rakam++;
43          } else if (isspace(c)) {
44              bosluk++;
45          } else {
46              diger++;
47          }
48      }
```

4.2 Örnek 6–7: ters çevirme ve palindrom (★★)

Örnek 6 — yerinde ters çevirme. H10'un iki uç tekniği string'e uygulanır: `i` baştan, `j` sondan; `s[i]` ile `s[j]` takas edilir, `i < j` oldukça. `'\0'` yerinde kalır; uzunluk `strlen` ile bulunur.

H13_06_ters_cevir.c · satır 16–31

```
16  void ters_cevir(char *s)
17  {
18      size_t i = 0;
19      size_t j = strlen(s);
20      if (j == 0) {
21          return;                     /* boş string: yapılacak bir şey yok */
22      }
23      j--;                            /* son karakterin indisi ('\0'dan önce) */
24      while (i < j) {
25          char gecici = s[i];         /* üç atamayla takas (H3) */
26          s[i] = s[j];
27          s[j] = gecici;
28          i++;
29          j--;
30      }
31  }
```

Gerçek çıktı: “"merhaba" -> "abahrem"”, “"AP-1 2026" -> "6202 1-PA"”, “"x" -> "x"”. Bu **bayt bazlı** ters çevirmedir: Türkçe çok baytlı harfler baytlarına ayrılacağından yalnızca ASCII metinler doğru çevrilir (Bölüm 3.10). **Örnek 7 — palindrom.** Karşılaştırmadan önce ÖN İŞLEME yapılır: harf olmayanlar atlanır, harfler küçük harfe çevrilir; sonra iki uç karşılaştırılır (“Ey Edip Adana da pide ye” → palindromdur; “Merhaba dünya” → değildir):

H13_07_palindrom.c · satır 23–39

```
23      while (i < j) {
24          while (i < j && !isalpha((unsigned char)s[i])) {
25              i++;                    /* soldaki harf olmayanı atla */
26          }
27          while (i < j && !isalpha((unsigned char)s[j - 1])) {
28              j--;                    /* sağdaki harf olmayanı atla */
29          }
30          if (i < j) {
31              int sol = tolower((unsigned char)s[i]);
32              int sag = tolower((unsigned char)s[j - 1]);
33              if (sol != sag) {
34                  return false;
35              }
36              i++;
37              j--;
38          }
39      }
```

4.3 Amiral örnek — Örnek 9 ★★: Sezar şifreleme ve kaba kuvvet

1. Problem. Klasik Sezar şifresiyle bir metni şifreleyin ve çözün; anahtar bilinmiyorsa 26 anahtarın hepsini deneyerek (kaba kuvvet) kırmaya çalışın. Bu, karakter işleme ile siber güvenliğin buluştuğu örnektir.

Tablo 13.3
Örnek 9'un girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• metin (İngiliz alfabesi) • anahtar k (0–25)	• her harfi (A-Z, a-z) k kadar kaydır (mod 26) • harf olmayanlara dokunma • çözme: (26 − k) ile kaydır	• şifreli/çözülmüş metin • kaba kuvvette 26 aday

2–3. Çözümleme ve algoritma. Kaydırma, harfin alfabedeki **sırasına** dayanır (`c - 'A'`); bu yalnızca A–Z gibi ardışık ve tek baytlık kodlarda işler. **Neden İngiliz alfabesi?** Türkçe harfler UTF-8'de çok baytlı ve alfabetik olarak ardışık değildir (Bölüm 3.10); bu yüzden yalnızca ASCII harfler kaydırılır, Türkçe harfler olduğu gibi kalır. Modüler aritmetik başa sarmayı sağlar: `(c - 'A' + k) % 26`. **4. C kodu:**

H13_09_sezar.c · satır 20–31

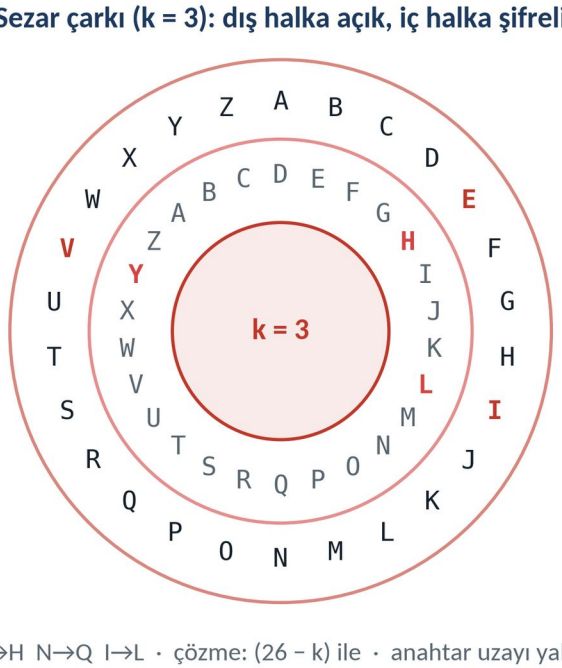
```c
void sezar(char *s, int k)
{
    k = ((k % 26) + 26) % 26;        /* negatif anahtar da çalışsın */
    for (size_t i = 0; s[i] != '\0'; i++) {
        unsigned char c = (unsigned char)s[i];
        if (c >= 'A' && c <= 'Z') {
            s[i] = (char)('A' + (c - 'A' + k) % 26);
        } else if (c >= 'a' && c <= 'z') {
            s[i] = (char)('a' + (c - 'a' + k) % 26);
        }                            /* harf değilse dokunma */
    }
}
```

5. Satır satır. `k = ((k % 26) + 26) % 26` negatif anahtarı da 0–25'e indirger (çözme için `-k` verilebilir). Döngü her baytı `(unsigned char)` ile okur; büyük harf aralığındaysa `'A'` tabanıyla, küçük harf aralığındaysa `'a'` tabanıyla kaydırır; harf değilse dokunmaz. **6. İzleme (k = 3).**

Tablo 13.4
"VENI" → "YHQL" izlemesi (k = 3)

Harf	c - taban	+k	% 26	Sonuç
V	21	24	24	**Y**
E	4	7	7	**H**
N	13	16	16	**Q**
I	8	11	11	**L**

Şekil 13.9
Sezar çarkı (k = 3): dış halka açık harf, iç halka şifreli; anahtar uzayı yalnızca 26



7. Örnek çalıştırma (gerçek):

GERÇEK ÇIKTI (ŞİFRELEME, ÇÖZME VE KABA KUVVETİN BİR KISMI)

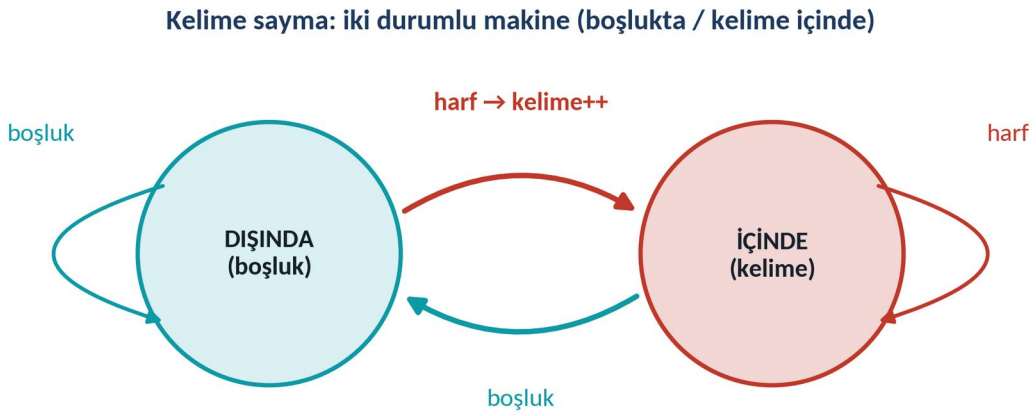
```
Açık metin : VENI VIDI VICI
k = 3 şifre : YHQL YLGL YLFL
Çözülmüş   : VENI VIDI VICI
...
  k= 0: YHQL YLGL YLFL
  k= 1: XGPK XKFK XKEK
  k= 2: WFOJ WJEJ WJDJ
  k= 3: VENI VIDI VICI
```

8. Test. Şifrele-sonra-çöz, özgün metni geri verir (`sezar(s, 3)` ardından `sezar(s, -3)` → "VENI VIDI VICI"). **9. Değiştir.** (a) Anahtarı kullanıcıdan `strtol` ile okuyun (Örnek 15). (b) Kaba kuvvette hangi satırın anlamlı olduğunu insan seçer; bir sonraki adım (Örnek 14) bunu **frekans analiziyle otomatikleştirir**.

4.4 Örnek 13–14: kelime sayma ve frekans analizi (★★★)

Örnek 13 — kelime sayma. Metin `stdin`'den `EOF`'a kadar `getchar` ile okunur; kelime sayımının özü bir **durum makinesidir** (Şekil 13.10): her an "kelime içinde" ya da "dışında" (boşlukta) oluruz, yalnızca dıştan içe **geçişte** sayaç artar.

Şekil 13.10
Kelime sayma durum makinesi: yalnızca boşluktan harfe geçişte kelime sayılır



Yalnızca 'dışından içeri' geçişte sayaç artar (yeni kelime başlangıcı)

H13_13_kelime_sayma.c · satır 24–35

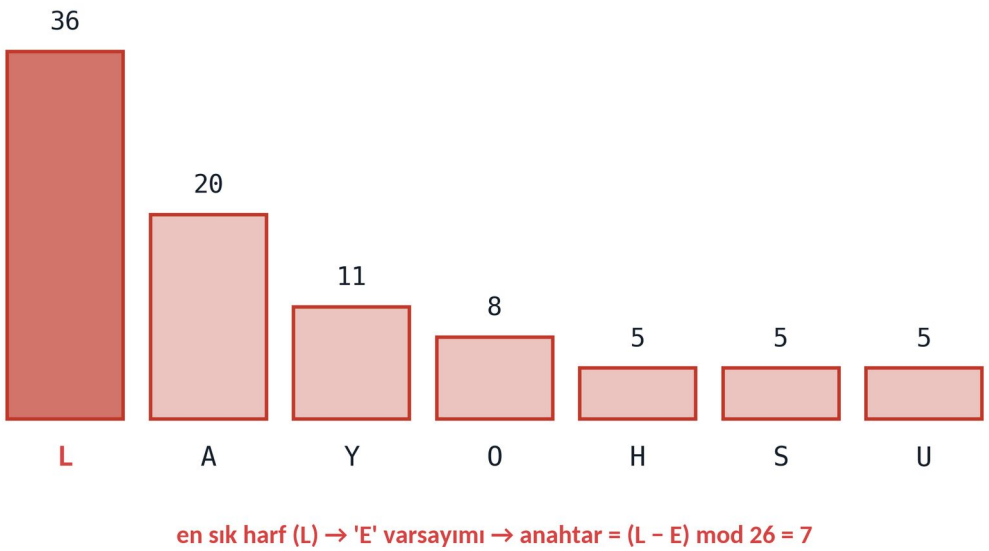
```c
    while ((c = getchar()) != EOF) {
        bayt++;                 /* getchar bir BAYT okur */
        if (c == '\n') {
            satir++;
        }
        if (isspace(c)) {
            kelime_icinde = 0;   /* boşluk: kelime dışına çık */
        } else if (!kelime_icinde) {
            kelime_icinde = 1;   /* dıştan içe GEÇİŞ: yeni kelime */
            kelime++;
        }
    }
```

Gerçek çalıştırma (`printf 'bir iki üç\ndört beş\n' | ./kelime_sayma`): Bayt: 24, Kelime: 5, Satır: 2 — `getchar` BAYT okuduğundan ilk sayı bayttır: 20 karakterlik girdi, ü, ç, ö ve ş ikişer bayt olduğundan 24 bayttır (Örnek 12). **Örnek 14 — frekans analizi (meydan okuma).** H10'un frekans dizisi (`sayac[c - 'A']`) metne uygulanır ve bununla bir Sezar şifresi **anahtar bilinmeden** kırılır (Şekil 13.11): İngilizcede en sık harf 'E' olduğundan, şifreli metnin en sık harfinin 'E'ye karşılık geldiği varsayılır: `anahtar = (en_sik - 'E' + 26) % 26`. Sayım H10'un frekans deseniyle yapılır:

H13_14_frekans_sezar_kirma.c · satır 42–48

```c
    int sayac[26] = {0};      /* harfin kendisi indis: H10 frekans deseni */
    for (size_t i = 0; sifre[i] != '\0'; i++) {
        unsigned char c = (unsigned char)toupper((unsigned char)sifre[i]);
        if (c >= 'A' && c <= 'Z') {
            sayac[c - 'A']++;
        }
    }
```

Şekil 13.11
Şifreli metnin harf frekansı: en sık harf (L) → 'E' varsayımı → anahtar 7 (gerçek çıktı)



Gerçek çıktı: “En sık harf: L -> 'E' varsayımı -> tahmini anahtar: 7” ve metin doğru çözülür. Frekans ve sayma, AP-2'de karma (hashing) ve sayma sıralaması sezgisinin ilk adımıdır (Bölüm 10).

4.5 Amiral örnek — Örnek 16 ★★★: tampon taşması ve güvenli sürüm

1. Problem. Kullanıcıdan bir ad alıp “Merhaba, <ad>!” yazan bir fonksiyon yazın. Güvensiz bir sürüm ve güvenli sürümünü yan yana çalıştırıp farkı **gerçek araç çıktılarıyla** gösterin. *(Mekanizma kavramsaldır; bir saldırı tarifi değildir.)*

Tablo 13.5
Örnek 16'nın girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• kullanıcıdan bir ad (uzunluğu bilinmez)	• güvensiz: 8 baytlık yığın tamponuna strcpy • güvenli: fgets ile boyut sınırlı okuma + snprintf	• güvensizde: kısa ad çalışır, uzun ad taşırır • güvenlide: uzun ad güvenle kesilir

2–3. Çözümleme. Sorun, tamponun boyutunun sabit (8 bayt) ama girdinin uzunluğunun bilinmemesidir. Girdi yürütme zamanında geldiği için derleyici taşmayı önceden göremez: dosya uyarısız derlenir ama **güvenli değildir**. **4. Güvensiz kod (kasıtlı):**

H13_16_tampon_tasmasi.c · satır 27–32

```
27  void selamla(const char *isim)
28  {
29      char tampon[8];                 /* yalnızca 8 bayt: 7 karakter + '\0' */
30      strcpy(tampon, isim);           /* HATA: sınır denetimi yok, TAŞABİLİR */
31      printf("Merhaba, %s!\n", tampon);
32  }
```

5–7. Üç gözlem (gerçek çalıştırmalar, bu derleyici: gcc 13, x86-64 Linux). (a) **Kısa ad güvenli:** “Ali” → “Merhaba, Ali!”, program normal biter. (b) **Uzun ad, ASan'sız:** derleyici yığına koyduğu **kanarya** (`-fstack-protector`) bozulunca programı durdurur:

ASAN'SIZ: KANARYA KORUMASI

```
$ ./tampon_tasmasi      # girdi: AbdurrahmanGaziHan (uzun)
*** stack smashing detected ***: terminated
bash: line 1:  9683 Aborted                 ./tampon_tasmasi
çıkış durumu: 134
```

(c) **Uzun ad, AddressSanitizer ile:** hatayı ilk olduğu satırda, hangi değişkenin taştığıyla raporlar:

ADDRESSSANİTİZER: STACK-BUFFER-OVERFLOW (KISALTILMIŞ; TAMAMI CİKTİLAR/ ALTINDA)

```
ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7fbf7cf00028 at pc
0x7fbf7f4a7923 bp 0x7ffd55974960 sp 0x7ffd55974108
WRITE of size 19 at 0x7fbf7cf00028 thread T0
    #1 0x55c73169638c in selamla H13_16_tampon_tasmasi.c:30
    [32, 40) 'tampon' (line 29) <== Memory access at offset 40 overflows this
variable
SUMMARY: AddressSanitizer: stack-buffer-overflow
../../../../src/libsanitizer/asan/asan_interceptors.cpp:563 in strcpy
```

ASan raporu üç şeyi söyler: hata türü (**stack-buffer-overflow**), nerede yazıldığı (`selamla`, satır 30, `strcpy` içinde) ve hangi değişkenin taştığı (`'tampon'`, satır 29). **4′. Güvenli sürüm** (`H13_16b_guvenli_surum.c`): girdi tamponun kendi boyutuyla okunur ve kopyalama `snprintf` ile sınırlanır:

H13_16b_guvenli_surum.c · satır 23–32

```c
void selamla(const char *isim)
{
    char tampon[32];
    /* snprintf en çok sizeof tampon - 1 bayt yazar ve '\0' koyar: taşmaz */
    int gerekli = snprintf(tampon, sizeof tampon, "Merhaba, %s!", isim);
    if (gerekli >= (int)sizeof tampon) {
        printf("(uyarı: mesaj %d bayta sığmadı, kesildi)\n", gerekli);
    }
    printf("%s\n", tampon);
}
```

Güvenli sürümde uzun ad **güvenle kesilir**, taşma olmaz (“(not: ad 7 bayta sığmadı, kesildi)”, “Merhaba, Abdurra!”). Bir incelik: `fgets` satırda `'\n'` bulamadıysa ad ya TAM 7 karakterdir (`'\n'` akışta kalmıştır) ya da daha uzundur; program bir sonraki karakteri okuyarak ikisini ayırt eder (Mini-OBS `satir_oku` deseni). Bu yüzden “Mehmet7” kesilmez: “Merhaba, Mehmet7!”. **8–9. Test ve değiştir.** Kısa ve uzun adlarla her iki sürümü de deneyin; güvenli sürümü ASan altında çalıştırın (rapor yok). Hangi SEI CERT kuralı ihlal edilmişti? (STR31-C). Atölye 3'te `gets`/`strcpy`/`sprintf` kalıplarını güvenli karşılıklarına siz dönüştüreceksiniz (Bölüm 8).

5. Hata Avcısı

Bu haftanın altı sık yanılgısı ve düzeltmeleri aşağıdadır (kodlar `kodlar/hata_avcisi/`). Kasıtlı hatalı dosyalar başlıklarında “BU DOSYA KASITLI HATA İÇERİR” diye imlenir; derleyici/çalışma çıktıları `ciktilar/` altındadır.

Tablo 13.6
Hata Avcısı: hata | belirti | neden | çözüm

Hata	Belirti / ileti	Neden	Çözüm
HA1: `char kod[5] = "HELLO"`	sınır dışı okuma (ASan: stack-buffer-overflow)	5 harf diziyi doldurur, `'\0'` için yer yok	`char kod[] = "HELLO"` (boyut + 1)
HA2: `girdi == dogru`	içerik aynıyken “farklı” der	`==` adresleri karşılaştırır (H12)	`strcmp(girdi, dogru) == 0`
HA3: `scanf("%s", ad)`	uzun girdide taşma / stack smashing	genişlik yok: sınırsız okuma	`scanf("%7s", ad)` ya da `fgets`
HA4: `strcmp(cevap, "evet")`	“evet” girilse de eşleşmez	fgets sondaki `'\n'`i bıraktı	`cevap[strcspn(cevap,"\n")]='\0'`
HA5: `strtok("a,b,c", ...)`	tanımsız davranış; bu sistemde Segmentation fault (139)	strtok değişmeze `'\0'` yazar	değiştirilebilir diziye kopyala
HA6: `char *s="abc"; s[0]='X'`	tanımsız davranış; bu sistemde Segmentation fault (139)	string değişmezi salt okunur	`char s[] = "abc";`

İki örnek yakından. HA4 (fgets '\n' tuzağı) derlenir ve çalışır ama yanlış yanıt verir: “evet” girildiğinde “(yanlış) Onaylanmadı”; düzeltilmiş sürüm `strcspn` ile '\n'i siler ve “Onaylandı” der. HA5 ise bir string

DEĞİŞMEZİNİ `strtok`'a verir; `strtok` ayraç yerine '\0' yazmaya çalışır; bu **tanımsız davranıştır** ve bu sistemde program çöktü (çıkış durumu 139); düzeltme, metni `char csv[] = "..."` ile değiştirilebilir bir diziye kopyalamaktır. İki hata da aynı dersi verir: string'ler bayt dizileridir; nereye yazdığınızı ve o belleğin yazılabilir olup olmadığını bilmelisiniz.

6. Mühendis Gözüyle: Güvenli Kodlama, Araçlar ve Taşınabilirlik

Metin işleme, C'de güvenlik açıklarının en yoğun olduğu alandır: girdinin uzunluğu önceden bilinmez, tamponlar sabittir ve `'\0'` kolayca unutulur. Profesyonel C kodunda dört alışkanlık riski büyük ölçüde azaltır:

- **Boyut her zaman bilinir ve sınırlanır.** Her tampon için boyutu ayrı tutun; her okuma ve kopyalama bu boyutla sınırlansın: `fgets(b, sizeof b, stdin)`, `snprintf(b, sizeof b, ...)`, `strncpy` + elle `'\0'`. `gets` asla kullanılmaz (C11'de kaldırıldı).
- **Her string sonlandırıcıyla biter** (`'\0'`). `strncpy`'den sonra son baytı elle sonlandırın; `char kod[] = "..."` ile boyutu derleyiciye saydırın (boyut + 1).
- **Değişmezler salt okunurdur.** Değiştirilecek metin bir diziye kopyalanır; string değişmezine yazılmaz (STR30-C).
- **Araçla doğrulama.** Metin ve bellek kullanan her değişiklik `-fsanitize=address,undefined` ve (Linux/WSL'de) `valgrind` ile denenir; kanarya koruması (`-fstack-protector`) açık bırakılır.

Bu haftanın hataları SEI CERT C kodlama standardının kurallarıyla doğrudan eşleşir (Carnegie Mellon University Software Engineering Institute, 2016; Seacord, 2013): **STR30-C** string değişmezini değiştirmeyin (HA6, K01); **STR31-C** string için yeterli yer (ve `'\0'`) ayırın (HA1, HA3, Örnek 16); **STR32-C** kütüphane fonksiyonlarına `'\0'` ile bitmeyen dizi vermeyin (K06); **STR37-C** karakter işleme fonksiyonlarının argümanı `unsigned char` olarak gösterilebilmeli (K09, tüm örnekler); **ERR34-C** metni sayıya çevirirken hataları saptayın (`strtol`, K10, Örnek 15). İlgili açık sınıfları: **CWE-120** (girdi boyutunu denetlemeden kopyalama) ve **CWE-121** (yığın tabanlı tampon taşması).

Taşınabilirlik. Kaynak dosyayı UTF-8 kaydedin; GCC'nin varsayılan kaynak/yürütme kodlaması UTF-8'dir. Windows konsolunda Türkçe karakterler için `chcp 65001` gerekir. `char`'ın işaretli mi işaretsiz mi olduğu derleyiciye bağlıdır; bu yüzden `ctype` ve bayt işlemlerinde her zaman `(unsigned char)` dönüşümü yapılır. AddressSanitizer, MinGW-w64'ün GCC'sinde yoktur (WSL ya da Visual Studio kullanılır); Valgrind yalnızca Linux/WSL'de çalışır. **Verimlilik.** `strlen`'i bir döngü koşulunda tekrar tekrar çağırmak (`for (i = 0; i < strlen(s); i++)`) her yinelemede string'i baştan tarar (n karakterlik bir string için yaklaşık n^2 adım); uzunluğu bir kez hesaplayıp saklayın.

7. Mini-OBS v0.13: Ad–Soyad, İsimle Arama ve Güvenli Girdi

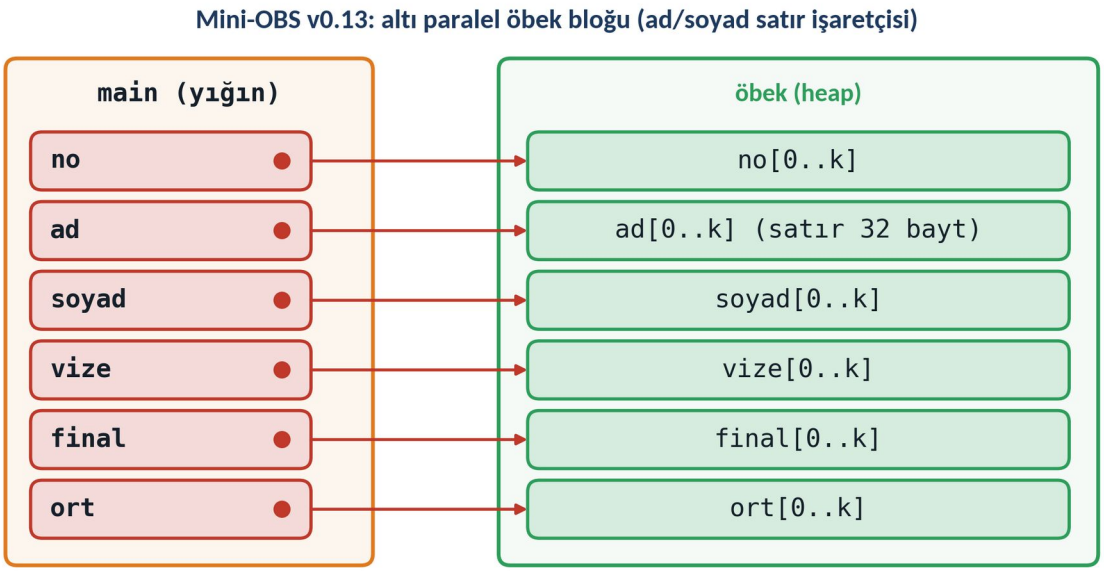
v0.12'de kayıtlar öbekte altı değil **dört** paralel bloktaydı (no, vize, final, ort) ve öğrenci yalnızca numarayla tutuluyordu. v0.13 iki yeni blok ekler — **ad ve soyad** — ve dönemin karakter dizisi araçlarını projeye taşır: isimle arama, ad normalleştirme ve güvenli girdi. Hesap kuralları (RTEÜ harf tablosu, final barajı 50, şartlı DC, iki haneye yuvarlanan YANO), istatistik ve harf tablosu biçimleri v0.12 ile **birebir aynıdır**; testler 79'dan 108 doğrulamaya çıktı (29 metin işleme testi eklendi).

Ad/soyad öbekte satır işaretçisiyle. Her ad, sabit uzunluklu bir SATIR olarak tutulur; bu, H12'deki `double (*vize)[DERS_SAYISI]` deseninin aynısıdır (Şekil 13.12):

proje/main.c · satır 33–34

```
33      char (*ad)[AD_BOYUT] = malloc(kapasite * sizeof *ad);
34      char (*soyad)[SOYAD_BOYUT] = malloc(kapasite * sizeof *soyad);
```

Şekil 13.12
Mini-OBS v0.13: main'in yığınındaki altı işaretçi, öbekteki altı bloğu gösterir; ad[i] i. öğrencinin '\0' ile biten adıdır



ad: char(*)[32] · notlar: double(*)[3] · kapasite dolunca altısı da realloc ile 2 katına

Altı paralel blok zahmeti → H14'te tek struct Ogrenci

`ad[i]`, i. öğrencinin `'\0'` ile biten karakter dizisidir; `sizeof *ad` bir satırın (32 bayt) boyutudur. Kapasite dolunca **altı blok da** `buyut` (realloc) ile ikiye katlanır (v0.12'nin dört bloklu deseni altıya genişler). **Yeni yetenekler:**

- `ad_normallestir` (obs.c): baştaki/sondaki boşlukları atar, aradaki boşluk dizilerini teke indirir ve harfleri Türkçe kurala göre büyütür (i → İ, ı → I, ç → Ç …) — hedef tampona asla taşmadan ve çok baytlı bir harfi asla ortasından bölmeden. Türkçe büyük harf, UTF-8 farkında bir eşleme tablosuyla yapılır (Bölüm 3.10'daki "Türkçe I sorunu"nun çözümü).
- `utf8_karakter_sayisi` ve `hizali_yaz`: `printf`'in `%-24s`'i BAYT saydığı için Türkçe adlar sütunları kaydırır; `hizali_yaz` KARAKTER sayarak hizalar (listelerde ad soyad sütunu doğru dizilir).
- `isim_bul` (strcmp ile doğrusal tarama, `baslangic` parametresiyle bütün eşleşmeleri gezer): aranan ad normalleştirilir, sonra adla, soyadla ya da "AD SOYAD" tam adıyla eşleşen kayıtlar bulunur.
- **Güvenli girdi**: bütün girdiler `satir_oku` (fgets; taşan satırın atılması; CRLF temizliği) + `tamsayi_coz`/`ondalik_coz` (strtoll/strtod, endptr, ERANGE, `isfinite`, "85,5" reddi) ile okunur; `scanf` tamamen kaldırıldı.

Türkçe I sorunu gerçek test (T2). Kayıtlı soyad "YILMAZ" (noktasız I) iken "yilmaz" araması `ad_normallestir` ile "YİLMAZ"a (noktalı İ) normalleşir ve **eşleşmez**: gerçek çıktı "Eşleşen kayıt yok: YİLMAZ". Oysa zaten büyük yazılmış "YILMAZ" araması iki kaydı bulur. Bu, Türkçe büyük harf kuralının İngilizceden farkını somutlaştırır (bilinçli bir sınırdır; `mini_obs_v0_13.md`, Bölüm 8).

Test ve bellek denetimi (gerçek). `make test`: "Tüm testler geçti (108 doğrulama: 37 kural + 14 dizi + 10 matris + 9 işaretçi + 9 bellek + 29 metin).". `make asan` (ASan + UBSan + LSan): T1–T6 altında rapor

yok. `make valgrind`: her testte "All heap blocks were freed -- no leaks are possible" (ör. T2 total heap usage: 23 allocs, 23 frees, 10,328 bytes allocated). `make bellek_siniri`: 1 MiB ayırma sınırıyla büyük kapasitede realloc **gerçekten başarısız** olur: "Büyük kapasite: 30000 -> 30000 (realloc NULL döndü; kayıtlar korundu)". v0.12 ile aynı notlarla çalıştırıldığında **ders istatistikleri ve harf tablosu bölümleri birebir aynıdır** (hesap kuralları değişmedi; karşılaştırma `mini_obs_v0_13_ciktilar.txt`'de). Ayrıntılar ve kabul ölçütleri proje klasöründeki `mini_obs_v0_13.md` belgesindedir.

8. Alıştırmalar ve Atölye

Kodlar `kodlar/alistirmalar/` altındadır (gösterilen kesitlerin satır numaraları dosyalarla aynıdır; `#include` satırları kesitlerde gösterilmedi). Çözümler öğretim üyesindedir (92_Degerlendirme).

8.1 Alıştırmalar

Alıştırma 13.1 ★☆☆ *Çıktı tahmini*

Program ne yazdırır? `sizeof`, `strlen` ve `'\0'`a kadar sayan döngü arasındaki farkı düşünün.

`H13_A01_cikti_tahmini.c · satır 13–19`

```
13      char s[10] = "AP-1";
14      int sayac = 0;
15      for (int i = 0; s[i] != '\0'; i++) {
16          sayac++;
17      }
18      printf("%zu %zu %d\n", sizeof s, strlen(s), sayac);
19      return 0;
```

Alıştırma 13.2 ★★☆ *Hata bul*

`tam_ad[8]` tamponu "Ayse Yilmaz" (11 karakter + `'\0'`) için çok küçük; `strcpy`/`strcat` taşırır. Hatayı bulun ve güvenli biçimde (boyut + 1; `snprintf`) düzeltin.
(`H13_A02_hata_bul_birlestir.c`; kasıtlı taşma içerir.)

Alıştırma 13.3 ★★☆ *Parsons*

`unlu_say(s)`, s'deki ASCII ünlü (a,e,i,o,u; büyük/küçük) sayısını döndürmelidir. Karışık verilen satırları doğru sıraya dizin ve girintileyin. (`H13_A03_parsons_unlu_say.c`)

Alıştırma 13.4 ★★☆ *Kod tamamla*

`benim_strlen` ve `ters_yazdir` fonksiyonlarındaki iki boşluğu (`____`) doldurun: (1) `'\0'`a kadar ilerleyen indis, (2) sondan başa giden azaltma. (`H13_A04_kod_tamamla_ters.c`)

Alıştırma 13.5 ★★☆ *Çıktı tahmini*

k = 3 ile kaydırmada "AX BY" ne olur? `'X' + 3` alfabenin sonunda başa sarar mı? (boşluk değişmez)

`H13_A05_cikti_tahmini_sezar.c · satır 11–19`

```c
    char s[] = "AX BY";
    int k = 3;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') {
            s[i] = (char)('A' + (s[i] - 'A' + k) % 26);
        }
    }
    printf("%s\n", s);              /* ? */
    return 0;
```

Alıştırma 13.6 ★★☆ *Hata bul*

`fgets` ile okunan şehir adının sondaki `'\n'`i temizlenmediği için "İzmir" ile hiç eşleşmez. Hatayı bulun ve `strcspn` ile düzeltin. (`H13_A06_hata_bul_girdi.c`)

Alıştırma 13.7 ★★★ *Kod yaz*

Kendi strcat'imiz. `benim_strcat(hedef, kaynak)`, hedefin sonundaki `'\0'`ı bulup kaynağı oradan itibaren `'\0'` dahil kopyalasın. Hedefin yeterli yere sahip olduğunu varsayın (STR31-C) ve `hedef`'i döndürün.

Alıştırma 13.8 ★★★ *Kod yaz*

Baş harfleri büyüt. `bas_harf_buyut(s)`, her kelimenin baş harfini büyük, kalanını küçük yapsın (yalnızca ASCII; durum makinesi: boşluktan harfe geçişteki ilk harf). "merhaba DUNYA" → "Merhaba Dunya".

8.2 Atölye etkinlikleri

Atölye 1 — Metin çözümleme aracı (wc benzeri)

`stdin`'den `EOF`'a kadar okuyup karakter, kelime ve satır sayısını veren bir araç yazın (Unix `wc` benzeri). Kelime sayımı bir durum makinesidir (Örnek 13). İpucu: `getchar` BAYT okur; Türkçe harfli bir girdide karakter sayınızı Örnek 12'nin bayt/karakter ayrımıyla karşılaştırın. Genişletme: en uzun satırı ve her harfin frekansını da raporlayın. Çözüm öğretim üyesindedir.

Atölye 2 — Sezar şifre makinesi ve frekans analiziyle kırma

Örnek 9 ve 14'ü birleştirin: bir metni anahtarla şifreleyin, sonra anahtarı bilmeden hem 26 anahtarı deneyerek (kaba kuvvet) hem de frekans analiziyle (en sık harf → 'E') kırın. Çözüm öğretim üyesindedir. Sınıf yarışması: kim daha uzun bir metni daha hızlı kırar? Tartışma: anahtar uzayı neden yalnızca 26? (Bu, modern şifrelemenin neden çok daha büyük anahtarlar kullandığını sezdirir.)

Atölye 3 — Güvensiz koddan güvenli koda

Verilen güvensiz kalıpları güvenli karşılıklarına dönüştürün: `gets` → `fgets` (boyut sınırlı), `strcpy` → boyut denetimli kopya, `sprintf` → `snprintf` (kesilme denetimli). Hiçbir yazma hedef tampon

boyutunu aşmamalı (STR31-C). Her dönüşümün hangi CERT kuralını karşıladığını yazın; çözüm öğretim üyesindedir. Kendi Örnek 16'nızı ASan ile doğrulayın: güvensiz sürüm rapor üretmeli, güvenli sürüm üretmemeli.

9. Kendini Değerlendirme

- ☐ Bir string'in `'\0'` ile biten bir char dizisi olduğunu ve "boyut + 1" kuralını açıklayabiliyorum.
- ☐ `sizeof` ile `strlen` farkını örnekle gösterebiliyorum.
- ☐ `scanf("%s")` yerine neden `fgets` kullanıldığını ve sondaki `'\n'`i nasıl temizlediğimi biliyorum.
- ☐ `strlen`, `strcpy`, `strcmp`'yi kendim yazabiliyor ve `strncpy`/`strtok` tuzaklarını tanıyorum.
- ☐ Karakter sayma, ters çevirme, palindrom, Sezar ve kelime sayma algoritmalarını tasarlayabiliyorum.
- ☐ `ctype` fonksiyonlarına argümanı neden `(unsigned char)` ile verdiğimi açıklayabiliyorum.
- ☐ UTF-8'de bayt ≠ karakter olduğunu ve `strlen("çay") == 4` olduğunu biliyorum.
- ☐ `argc`/`argv` ile komut satırı argümanlarını işleyebiliyorum.
- ☐ Tampon taşması mekanizmasını ve üç önleme katmanını (boyut sınırı, kanarya, ASan) açıklayabiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · Bu haftanın AP-2'ye tohumları

- **strcmp → string sıralaması.** İki metni sözlük sırasına göre karşılaştırmak, AP-2'de string dizilerini sıralamanın temelidir; karşılaştırma fonksiyonu `qsort`'a verilir (H12 fonksiyon işaretçisi).
- **strstr → desen eşleme.** Bir metinde alt string aramak, AP-2'de desen eşleme algoritmalarına (naif yöntem, Knuth–Morris–Pratt) giriş noktasıdır; `strstr` naif yöntemin kütüphane sürümüdür.
- **Harf frekansı → karma (hashing).** Karakterin kendisini indis yapmak (`sayac[c]`), AP-2'deki karma tablosu ve sayma sıralaması sezgisinin ilk adımıdır.

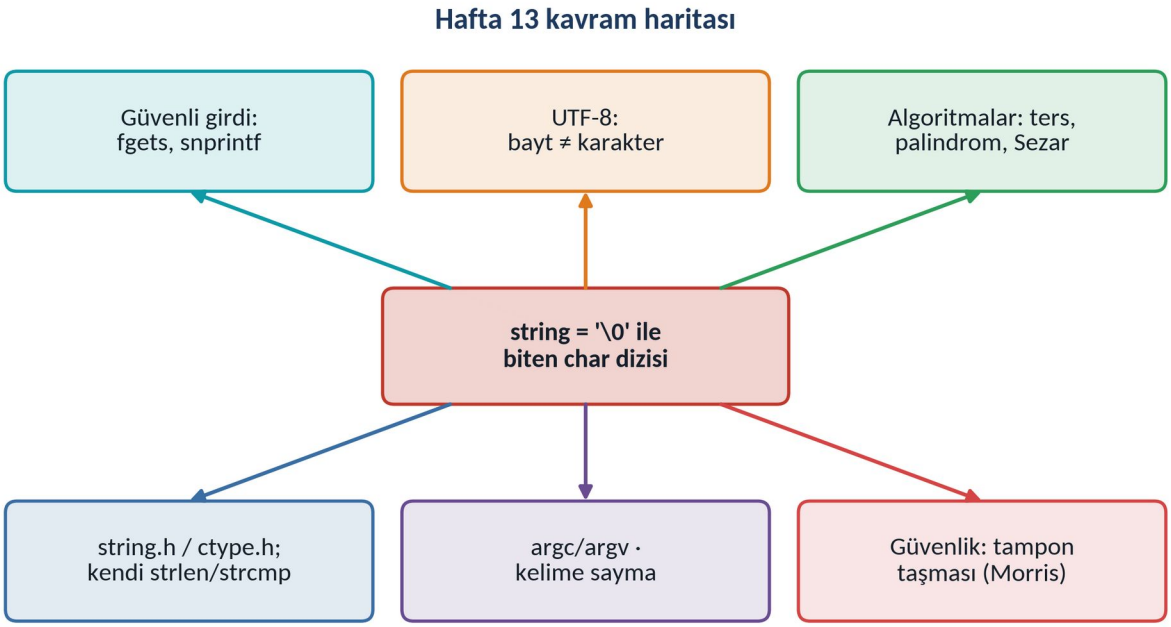
MERAK KUTUSU · i[a] ve diğer tuhaflıklar

H12'de `a[i]`'nin `*(a + i)` olduğunu görmüştük; toplama değişmeli olduğundan `i[a]` de (yani `2["abc"]`) geçerli C'dir ve `'c'` verir — ama yalnızca bir meraktır, hiç kullanılmaz. Daha yararlı bir merak: `strtok_r` (POSIX), `strtok`'un durum tutmayan, çok iş parçacıklı güvenli sürümüdür. Ve `wchar_t`/`<wchar.h>`, karakteri tek bir geniş birimde tutan ayrı bir dünyadır; UTF-8 ile çoğu zaman gerekmez ama metnin karakter karakter (bayt değil) işlenmesi gerektiğinde gündeme gelir.

11. Özet, Kavram Haritası ve Terimler

Bu hafta string'in `'\0'` ile biten bir char dizisi olduğunu bellek düzeyinde gördük; güvenli girdiyi (`fgets`, `snprintf`), `string.h`/`ctype.h`'yi ve temel fonksiyonların kendi gerçeklemelerini öğrendik; karakter düzeyinde algoritmalar (sayma, ters çevirme, palindrom, Sezar, kelime sayma, frekans analizi) tasarladık; UTF-8'de bayt ile karakterin farkını ve `argc`/`argv`'yi işledik; ve tampon taşmasının mekanizmasıyla önlemesini gerçek araç çıktılarıyla gördük.

Şekil 13.13
Hafta 13 kavram haritası: '\0' ile biten char dizisi ve altı dalı



Terim sözlüğü (İngilizce — Türkçe).

Tablo 13.7
Hafta 13 terim standardı

İngilizce	Türkçe	Not
string; null terminator	karakter dizisi; sonlandırıcı ('\0')	
buffer overflow	tampon taşması	CWE-120/121
character encoding; UTF-8	karakter kodlaması; UTF-8	Türkçe harfler 2 bayt
command-line arguments	komut satırı argümanları	argc, argv
state machine	durum makinesi	kelime sayma
frequency analysis	frekans analizi	Sezar kırma
secure coding	güvenli kodlama	SEI CERT C
stream	akış	stdin/stdout/stderr (H14)

12. Kaynaklar

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Carnegie Mellon University Software Engineering Institute. (2016). *SEI CERT C coding standard: Rules for developing safe, reliable, and secure systems* (2016 ed.). Carnegie Mellon University.

Deitel, P., & Deitel, H. (2022). *C how to program: With case studies introducing applications and systems programming* (9th ed., global ed.). Pearson.

Federal Bureau of Investigation. (2018, 2 Kasım). *The Morris worm: 30 years since first major attack on the Internet*. https://www.fbi.gov/news/stories/morris-worm-30-years-since-first-major-attack-on-internet-110218

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton.

Regents of the University of California. (1985). *fingerd.c* (Sürüm 5.1) [Kaynak kod, 4.3BSD]. Unix History Repository. https://github.com/dspinellis/unix-history-repo/blob/BSD-4_3/usr/src/etc/fingerd.c

Seacord, R. C. (2013). *Secure coding in C and C++* (2nd ed.). Addison-Wesley.

Spafford, E. H. (1989). The Internet worm program: An analysis. *ACM SIGCOMM Computer Communication Review*, *19*(1), 17–57. https://doi.org/10.1145/66093.66095